



Foundation University Islamabad

School of Science and Technology

Name: Hunaina Yasir

Subject: OOPS LAB

Roll Number: 073

Section: B

LAB REPORT NO 3

TASK 1:

Design a class named Student to store and manage student information using encapsulation. The class should have private data members: rollNo (an integer representing the student's roll number), name (a string for the student's name), and marks (a floating-point number representing the student's total marks). The class should include a getData() function to take user input for all data members and a showData() function to display the student's details. Additionally, implement a setData(int roll, string n, float m) function to set the values of rollNo, name, and marks using parameters, and a getMarks() function that returns the total marks. In the main() function, create a Student object, take input from the user, display the details, modify the marks using setData(), and print the updated marks.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    int rollNo;
    string name;
    float marks;
public:
    void getData() {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cin.ignore(); // Clear the buffer so getline works
        cout << "Enter Name: ";
        getline(cin, name);
        cout << "Enter Marks: ";
        cin >> marks;
    }

    void setData(int roll, string n, float m) { // Function to set data via parameters
        rollNo = roll;
```

```

        name = n;
        marks = m;
    }

    float getMarks() {
        return marks;
    }

    void showData() {
        cout << "\nStudent Details:" << endl;
        cout << "Roll No: " << rollNo << "\nName: " << name << "\nMarks: " << marks << endl;
    }
};

int main() {
    Student s1;

    // user entering data
    s1.getData();
    s1.showData();
    s1.setData(101, "Alex", 95.5); // Overwriting the data

    cout << "Updated Marks: " << s1.getMarks() << endl;

    return 0;
}

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Student {
6  private:
7      int rollNo;
8      string name;
9      float marks;
10
11 public:
12     void getData() {
13         cout << "Enter Roll Number: ";
14         cin >> rollNo;
15         cin.ignore(); // Clear the buffer so getline works
16         cout << "Enter Name: ";
17         getline(cin, name);
18         cout << "Enter Marks: ";
19         cin >> marks;
20     }
21
22     void setData(int roll, string n, float m) { // Function
23         rollNo = roll;
24         name = n;
25         marks = m;
26     }
27
28     float getMarks() {

```

```

22     void setData(int roll, string n, float m) { // Function to
23         rollNo = roll;
24         name = n;
25         marks = m;
26     }
27
28     float getMarks() {
29         return marks;
30     }
31
32     void showData() {
33         cout << "\nStudent Details:" << endl;
34         cout << "Roll No: " << rollNo << "\nName: "
35         << name << "\nMarks: " << marks << endl;
36     }
37 };
38
39 int main() {
40     Student s1;
41
42     // user entering data
43     s1.getData();
44     s1.showData();
45     s1.setData(101, "Alex", 95.5); // Overwriting the data
46
47     cout << "Updated Marks: " << s1.getMarks() << endl;
48
49     return 0;

```

OUTPUT:

Enter Roll Number: 73
Enter Name: Hunaina
Enter Marks: 900

Student Details:
Roll No: 73
Name: Hunaina
Marks: 900
Updated Marks: 95.5

```
Enter Roll Number: 73
Enter Name: Hunaina
Enter Marks: 900

Student Details:
Roll No: 73
Name: Hunaina
Marks: 900
Updated Marks: 95.5
```

TASK 2:

Create a class called BankAccount that represents a bank account using encapsulation. The class should contain private data members: accountNumber (an integer representing the unique account number) and balance (a double representing the current balance). Implement a deposit(double amount) function that adds the specified amount to the balance and a withdraw(double amount) function that deducts the amount from the balance if sufficient funds are available; otherwise, it should display an error message. The class should also have a getBalance() function to return the current balance and a display() function to show the account number and balance. In the main() function, create a BankAccount object, deposit an amount, attempt a withdrawal, and display the final balance.

SOURCE CODE:

```
#include <iostream>
using namespace std;

class BankAccount {
private:
    int accountNumber;
    double balance;

public:
    void setupAccount(int accNum, double initialBalance) {
        accountNumber = accNum;
        balance = initialBalance;
    }

    void deposit(double amount) {
        balance += amount;
        cout << "Deposited: $" << amount << endl;
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            cout << "Withdrew: $" << amount << endl;
        } else {
```

```

        // Check so the account doesn't go negative
        cout << "Error: Insufficient funds!" << endl;
    }
}

double getBalance() {
    return balance;
}

void display() {
    cout << "Account: " << accountNumber << " | Balance: $" << balance << endl;
}

};

int main() {
    BankAccount myAcc;
    myAcc.setupAccount(556677, 1000.0); // Start with a grand

    myAcc.deposit(500.0);
    myAcc.withdraw(200.0);
    myAcc.withdraw(2000.0); // This should trigger the error

    myAcc.display();
    return 0;
}

```

```

1  #include <iostream>
2  using namespace std;
3
4  class BankAccount {
5  private:
6      int accountNumber;
7      double balance;
8
9  public:
10 void setupAccount(int accNum, double initialBalance) {
11     accountNumber = accNum;
12     balance = initialBalance;
13 }
14
15 void deposit(double amount) {
16     balance += amount;
17     cout << "Deposited: $" << amount << endl;
18 }
19
20 void withdraw(double amount) {
21     if (amount <= balance) {
22         balance -= amount;
23         cout << "Withdrew: $" << amount << endl;
24     } else {
25         // Check so the account doesn't go negative
26         cout << "Error: Insufficient funds!" << endl;
27     }
28 }

```

```

22     balance -= amount;
23     cout << "Withdrew: $" << amount << endl;
24 } else {
25     // Check so the account doesn't go negative
26     cout << "Error: Insufficient funds!" << endl;
27 }
28
29
30 double getBalance() {
31     return balance;
32 }
33
34 void display() {
35     cout << "Account: " << accountNumber << " | Balance: $"
36     << balance << endl;
37 }
38 };
39
40 int main() {
41     BankAccount myAcc;
42     myAcc.setupAccount(556677, 1000.0); // Start with a grand
43
44     myAcc.deposit(500.0);
45     myAcc.withdraw(200.0);
46     myAcc.withdraw(2000.0); // This should trigger the error
47
48     myAcc.display();
49     return 0;

```

OUTPUT:

Deposited: \$500
Withdrew: \$200

Error: Insufficient funds!
Account: 556677 | Balance: \$1300

```
Deposited: $500
Withdrew: $200
Error: Insufficient funds!
Account: 556677 | Balance: $1300
```

TASK 3:

Develop a class named Car that encapsulates information about a car. The class should have private data members: brand (a string representing the car's brand), model (a string representing the car's model), and price (a double representing the car's price). Implement a setData(string b, string m, double p) function to assign values to the brand, model, and price. The class should also include a display() function to print the car's details and a getPrice() function to return the price of the car. In the main() function, create a Car object, initialize it using setData(), display the car's details, and retrieve and print the price using getPrice().

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Car {
private:
    string brand;
    string model;
    double price;

public:
    void setData(string b, string m, double p) {
        brand = b;
        model = m;
        price = p;
    }

    double getPrice() {
        return price;
    }

    void display() {
        cout << "Car: " << brand << " " << model << " | Price: $" << price << endl;
    }
};

int main() {
    Car myCar;
```

```

// Setting the car info directly
myCar.setData("Toyota", "Camry", 25000.0);
myCar.display();

cout << "The price tag is: $" << myCar.getPrice() << endl;

return 0;
}

```

```

2  #include <string>
3  using namespace std;
4
5  class Car {
6  private:
7      string brand;
8      string model;
9      double price;
10
11 public:
12     void setData(string b, string m, double p) {
13         brand = b;
14         model = m;
15         price = p;
16     }
17
18     double getPrice() {
19         return price;
20     }
21
22     void display() {
23         cout << "Car: " << brand << " " << model << " | Price: $" << price << endl;
24     }
25 };
26
27 int main() {
28     Car myCar;
29
30     // Setting the car info directly
31     myCar.setData("Toyota", "Camry", 25000.0);
32     myCar.display();
33
34     cout << "The price tag is: $" << myCar.getPrice() << endl;
35
36     return 0;
37 }

```

OUTPUT:

Car: Toyota Camry | Price: \$25000
The price tag is: \$25000

```

Car: Toyota Camry | Price: $25000
The price tag is: $25000

```

TASK 4:

Design a class named Employee that stores and manages employee details using encapsulation. The class should contain private data members: empID (an integer representing the employee's ID), name (a

string storing the employee's name), and salary (a double representing the employee's monthly salary). The class should include a getData() function to take input for all data members and a showData() function to display employee details. Additionally, implement a setSalary(double newSalary) function to update the salary and a getSalary() function to return the salary. In the main() function, create an Employee object, take input from the user, display the details, modify the salary using setSalary(), and print the updated salary.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
private:
    int empID;
    string name;
    double salary;

public:
    void getData() {
        cout << "Enter Employee ID: ";
        cin >> empID;
        cin.ignore();
        cout << "Enter Name: ";
        getline(cin, name);
        cout << "Enter Salary: ";
        cin >> salary;
    }

    void setSalary(double newSalary) {
        salary = newSalary;
    }

    double getSalary() {
        return salary;
    }

    void showData() {
        cout << "\nID: " << empID << "\nName: " << name << "\nSalary: $" << salary << endl;
    }
};

int main() {
    Employee emp;

    emp.getData();
    emp.showData();
```

```

// Giving the employee a raise
emp.setSalary(5000.0);

cout << "New Salary: $" << emp.getSalary() << endl;

return 0;
}

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Employee {
6  private:
7      int empID;
8      string name;
9      double salary;
10
11 public:
12     void getData() {
13         cout << "Enter Employee ID: ";
14         cin >> empID;
15         cin.ignore();
16         cout << "Enter Name: ";
17         getline(cin, name);
18         cout << "Enter Salary: ";
19         cin >> salary;
20     }
21
22     void setSalary(double newSalary) {
23         salary = newSalary;
24     }
25
26     double getSalary() {
27         return salary;
28     }

```

```

21
22     void setSalary(double newSalary) {
23         salary = newSalary;
24     }
25
26     double getSalary() {
27         return salary;
28     }
29
30     void showData() {
31         cout << "\nID: " << empID << "\nName: "
32         << name << "\nSalary: $" << salary << endl;
33     }
34 };
35
36 int main() {
37     Employee emp;
38
39     emp.getData();
40     emp.showData();
41
42     // Giving the employee a raise
43     emp.setSalary(5000.0);
44
45     cout << "New Salary: $" << emp.getSalary() << endl;
46
47     return 0;
48 }

```

OUTPUT:

```

Enter Employee ID: 34
Enter Name: leo
Enter Salary: 30000

```

```

ID: 34
Name: leo
Salary: $30000
New Salary: $5000

```

```

Enter Employee ID: 34
Enter Name: leo
Enter Salary: 30000

ID: 34
Name: leo
Salary: $30000
New Salary: $5000

```